

Задание 1.2

Установка и настройка программного обеспечения

Качков К.Е. - сделал скрипт.

Парванян Г.С. - протестировал скрипт, зайдя с нового пользователя.

Момот М.-М. М. - подготовила отчёт.

Скрипт начинается с определения минимальной версии оболочки, блока параметров и базовых настроек. Чтобы предотвратить незаметные сбои, обработка ошибок переводится в режим прерывания текущего потока. Для безопасного скачивания файлов по сети принудительно включается протокол TLS 1.2. В конце блока инициализируются пустые массивы для регистрации ошибок и создается папка в профиле пользователя для сохранения логов трассировки.

```
param(
    [Parameter(Mandatory = $true)]
    [ValidateSet(1, 2, 3)]
    [int]$Stage,
    [ValidatePattern('^[A-Za-z]:$')]
    [string]$BackupDrive = 'E:'
)

$errorActionPreference = 'Stop'if (Test-Path variable:PSNativeCommandUseErrorActionPreference)
{ $PSNativeCommandUseErrorActionPreference = $false }
[Net.ServicePointManager]::SecurityProtocol = [Net.ServicePointManager]::SecurityProtocol -bor
[Net.SecurityProtocolType]::Tls12
$script:Failures = @()
$script:RestartNeeded = $false
$folder = Join-Path $env:LOCALAPPDATA 'LabSetup'
$log = Join-Path $folder ('stage-{0}-{1}.log' -f $Stage, (Get-Date -Format 'yyyyMMdd-HH:mm:ss'))
New-Item -ItemType Directory -Path $folder -Force | Out-Null
```

Функция запуска изолированных задач перехватывает ошибки конкретных инсталляторов и сохраняет их в массив, позволяя скрипту продолжить установку остальных программ из списка. Функция запуска команд проверяет результаты работы внешних утилит и преобразует ненулевые коды возврата в исключения. Для корректной интеграции с WSL написана специальная обертка, которая читает консольный вывод в кодировке UTF-16, устраняя появление нечитаемых символов в логах Windows.

```
function Run-Task([string]$Name, [scriptblock]$Action) {
    Write-Host "`n>>> $Name" -ForegroundColor Cyan
    try { & $Action }
    catch {
        $script:Failures += "${Name}: $($_.Exception.Message)"
        Write-Warning $script:Failures[-1]
    }
}

function Run-Command([string]$File, [string[]]$Arguments = @()) {
    & $File @Arguments | Out-Host
    if ($LASTEXITCODE -ne 0) { throw "$File - код $LASTEXITCODE." }
}

function Run-Wsl([string[]]$Arguments, [switch]$Capture) {
    $start = New-Object System.Diagnostics.ProcessStartInfo
    $start.FileName = 'wsl.exe'
    $start.Arguments = $Arguments -join ' '
    $start.UseShellExecute = $false
    $start.CreateNoWindow = $true
    $start.RedirectStandardOutput = $true
    $start.RedirectStandardError = $true
    $start.StandardOutputEncoding = [Text.Encoding]::Unicode
}
```

```

$start.StandardErrorEncoding = [Text.Encoding]::Unicode
$start.EnvironmentVariables.Remove('WSL_UTF8')
$process = New-Object System.Diagnostics.Process
$process.StartInfo = $start
try {
    [void]$process.Start()
    $stdout = $process.StandardOutput.ReadToEndAsync()
    $stderr = $process.StandardError.ReadToEndAsync()
    $process.WaitForExit()
    $output = $stdout.Result
    $errorText = $stderr.Result
    if ($output) { Write-Host $output.TrimEnd() }
    if ($errorText) { Write-Host $errorText.TrimEnd() }
    if ($process.ExitCode -notin @(0, 3010)) {
        throw "wsl $($Arguments -join ' ') - код $($process.ExitCode). $errorText $output"
    }
    if ($process.ExitCode -eq 3010) {
        $script:RestartNeeded = $true
        if ($Stage -eq 2) { throw 'WSL требует перезагрузки. После нее повторите этап 2.' }
    }
    if ($Capture) { return @($output -split '\r?\n' | ForEach-Object { $_.Trim() } | Where-Object { $_ }) }
} finally { $process.Dispose() }
}
function Refresh-Path {
    $env:Path = [Environment]::GetEnvironmentVariable('Path', 'Machine') + ';' +
        [Environment]::GetEnvironmentVariable('Path', 'User')
}
function Add-UserPath([string]$Path) {
    if (-not (Test-Path -LiteralPath $Path -PathType Container)) { throw "Не найден каталог $Path." }
    $old = [string][Environment]::GetEnvironmentVariable('Path', 'User')
    if (@($old -split ';' | ForEach-Object { $_.TrimEnd('\') }) -notcontains $Path.TrimEnd('\')) {
        [Environment]::SetEnvironmentVariable('Path', ($old.TrimEnd(';') + ';' +
            $Path).TrimStart(';'), 'User')
    }
    Refresh-Path
}

```

Функция проверки программ анализирует системный реестр WinGet и корректно обрабатывает код отсутствия пакета в системе, возвращая логическое значение true или false. Функция установки автоматически принимает все лицензионные соглашения, подавляет графические окна инсталляторов и фиксирует специфические коды ошибок, требующие перезапуска компьютера. Функция загрузки скачивает бинарные файлы с официальных серверов, отключая стандартный индикатор прогресса для повышения скорости сетевого обмена, и проверяет первые байты полученных данных на соответствие форматам EXE или MSI для защиты от скачивания битых файлов.

```

function Test-Program([string]$Id) {
    & winget.exe list --id $Id --exact --source winget --accept-source-agreements --disable-
interactivity | Out-Host
    if ($LASTEXITCODE -eq 0) { return $true }
    if ($LASTEXITCODE -eq -1978335212) { return $false }
    throw "Не удалось проверить ${Id}: winget вернул $LASTEXITCODE."
}
function Install-Program([string]$Id) {
    if (Test-Program $Id) { Write-Host "Уже установлено: $Id"; return }
    & winget.exe install --id $Id --exact --source winget --silent --accept-package-agreements --
accept-source-agreements --disable-interactivity | Out-Host
    $code = $LASTEXITCODE
    if ($code -in @(3010, -1978334967, -1978334966)) {
        $script:RestartNeeded = $true
        throw 'Перезагрузите компьютер и повторите этап 2 для завершения установки и проверки.'
    }
}

```

```

    }
    if ($code -ne 0) { throw "Установка ${Id}: код $code." }
    if (-not (Test-Program $Id)) { throw "Установка $Id не подтверждена." }
}
function Download-Installer([string]$Url, [string]$Name) {
    $file = Join-Path $folder $Name
    $ProgressPreference = 'SilentlyContinue'
    Invoke-WebRequest -UseBasicParsing -Uri $Url -OutFile $file
    $stream = [IO.File]::OpenRead($file)
    try { $bytes = New-Object byte[] 8; $count = $stream.Read($bytes, 0, 8) }
    finally { $stream.Dispose() }
    $expected = if ($Name -like '*.msi') { 'D0-CF-11-E0-A1-B1-1A-E1' } else { '4D-5A' }
    if ($count -lt 8 -or -not ([BitConverter]::ToString($bytes).StartsWith($expected))) {
        throw "Вместо $Name получен файл неподходящего формата."
    }
    return $file
}

```

Функция установки WSL проверяет соответствие версии операционной системы минимальным требованиям, запрашивает через программный интерфейс GitHub метаданные последнего стабильного релиза подсистемы, скачивает инсталлятор, проверяет подлинность цифровой подписи Microsoft и запускает тихую установку пакета, после чего через DISM активирует платформу виртуальной машины. Функция проверки установленных приложений сканирует системные пути реестра на наличие программ по шаблону имени. Функция установки инструментов сборки проверяет наличие компонентов Visual Studio и дополняет конфигурацию пакетом компиляторов C++ для Rust, либо скачивает чистый инсталлятор Build Tools.

```

function Install-Wsl {
    if ([Environment]::OSVersion.Version.Build -lt 19041) { throw 'Для WSL нужны Windows 10 сборки 19041 или новее либо Windows 11.' }
    Write-Host 'WSL: установка из MSI Microsoft и включение Virtual Machine Platform.'
    $release = Invoke-RestMethod -Uri 'https://github.com'
    $asset = $release.assets | Where-Object { $_.name -match '^wsl\.[0-9.]+\x64\.msi$' } | Select-Object -First 1
    if (-not $asset) { throw 'В выпуске Microsoft WSL не найден установщик x64.' }
    Write-Host "Загрузка WSL $($release.tag_name)..."
    $setup = Download-Installer $asset.browser_download_url 'wsl-x64.msi'
    $signature = Get-AuthenticodeSignature -LiteralPath $setup
    if ($signature.Status -ne 'Valid' -or $signature.SignerCertificate.Subject -notmatch 'O=Microsoft Corporation') { throw 'Не подтверждена подпись Microsoft у WSL.' }

    $msilog = Join-Path $folder 'wsl-install.log'
    $arguments = '/i "{0}" /qn /norestart /L*v "{1}"' -f $setup, $msilog
    $process = Start-Process -FilePath 'msiexec.exe' -ArgumentList $arguments -WindowStyle Hidden -Wait -PassThru
    if ($process.ExitCode -notin @(0, 3010)) { throw "Установка WSL из MSI: код $($process.ExitCode). Журнал: $msilog" }
    if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }

    & dism.exe /Online /Enable-Feature /FeatureName:VirtualMachinePlatform /All /NoRestart /English
    | Out-Host
    if ($LASTEXITCODE -notin @(0, 3010)) { throw "Включение Virtual Machine Platform: код $LASTEXITCODE. Журнал: $env:windir\Logs\DISM\dism.log" }
    $script:RestartNeeded = $true
}
function Test-InstalledApp([string]$Pattern) {
    $keys = @(
        'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
        'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
        'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*'
    )
    return (@(Get-ItemProperty -Path $keys -ErrorAction SilentlyContinue |
        Where-Object { $_.DisplayName -match $Pattern }).Count -gt 0)
}
function Install-BuildTools {

```

```

$vsFolder = Join-Path ${env:ProgramFiles(x86)} 'Microsoft Visual Studio\Installer'
$vswhere = Join-Path $vsFolder 'vswhere.exe'
$installed = $null
if (Test-Path -LiteralPath $vswhere) {
    $installed = & $vswhere -latest -products Microsoft.VisualStudio.Product.BuildTools -
version '[17.0,18.0)' -property installationPath
    if ($LASTEXITCODE -ne 0) { throw 'Не удалось проверить Build Tools.' }
}
if ($installed) {
    $setup = Join-Path $vsFolder 'setup.exe'
    $arguments = 'modify --installPath "{0}" --channelId VisualStudio.17.Release --quiet --
norestart --add Microsoft.VisualStudio.Workload.VCTools --includeRecommended' -f
([string]$installed).Trim()
}
else {
    $setup = Download-Installer 'https://aka.ms' 'vs-buildtools.exe'
    $signature = Get-AuthenticodeSignature -LiteralPath $setup
    if ($signature.Status -ne 'Valid' -or $signature.SignerCertificate.Subject -notmatch
'O=Microsoft Corporation') { throw 'Не подтверждена подпись Microsoft у Build Tools.' }
    $arguments = '--quiet --wait --norestart --add Microsoft.VisualStudio.Workload.VCTools --
includeRecommended'
}
$process = Start-Process -FilePath $setup -ArgumentList $arguments -WorkingDirectory $folder -
WindowStyle Hidden -Wait -PassThru
if ($process.ExitCode -notin @(0, 3010)) { throw "Build Tools: код $($process.ExitCode)." }
if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }
}

```

Сценарий проверяет разрядность операционной системы и архитектуру процессора, после чего определяет контекст безопасности текущего пользователя. Здесь работает строгое разграничение прав доступа в зависимости от выбранной стадии: для этапов один и три скрипт требует обязательного запуска из терминала администратора, а для выполнения второго этапа администраторские права блокируются, требуя переключения на сессию обычного пользователя. Внутри условия первого этапа вызываются функции настройки подсистемы WSL и установки пакетов Build Tools, после чего скрипт останавливает поток выполнения для перезагрузки компьютера.

```

$exitCode = 0
Start-Transcript -Path $log | Out-Nulltry {
    if (-not [Environment]::Is64BitProcess -or $env:PROCESSOR_ARCHITECTURE -ne 'AMD64') { throw
'Нужны Windows x64 и 64-разрядный PowerShell.' }
    $principal = New-Object
Security.Principal.WindowsPrincipal([Security.Principal.WindowsIdentity]::GetCurrent())
    $admin = $principal.IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)
    if ($Stage -in @(1, 3) -and -not $admin) { throw 'Для этапов 1 и 3 откройте PowerShell от имени
администратора.' }
    if ($Stage -eq 2 -and $admin) { throw 'Для этапа 2 откройте обычный PowerShell под своей
учетной записью.' }

    if ($Stage -eq 1) {
        Install-Wsl
        Run-Task 'Build Tools для Rust' { Install-BuildTools }
        Write-Host 'Перезагрузите компьютер. Затем запустите этап 2 в обычном PowerShell.'
    }
}

```

Далее скрипт обновляет пути поиска команд сессии, проверяет доступность WinGet и переходит к конфигурированию WSL 2, устанавливая ее в качестве архитектуры по умолчанию. Программа проверяет список развернутых сред и принудительно переводит дистрибутивы Ubuntu версий 22.04 и 24.04 на вторую версию, а в случае их отсутствия скачивает дистрибутивы из сети без запуска графической оболочки. Затем объявляется массив из двадцати семи

программных пакетов, запускается цикл их последовательной установки через WinGet с изоляцией сбоев функцией Run-Task.

```
if ($Stage -eq 2) {
    Refresh-Path
    if (-not (Get-Command winget.exe -ErrorAction SilentlyContinue)) { throw
'Установите/обновите «Установщик приложений» Microsoft из Store и откройте PowerShell заново.' }

    Run-Task 'WSL 2' {
        Run-Wsl @('--version')
        Run-Wsl @('--set-default-version', '2')
        $installed = @(Run-Wsl @('--list', '--quiet') -Capture)
        foreach ($distro in @('Ubuntu-22.04', 'Ubuntu-24.04')) {
            if ($installed -contains $distro) { Run-Wsl @('--set-version', $distro, '2') }
            else { Run-Wsl @('--install', '-d', $distro, '--no-launch', '--web-download') }
        }
    }
    $programs = @(
        'Microsoft.VisualStudioCode', 'Docker.DockerDesktop', 'JetBrains.PyCharm.Community',
        'Git.Git', 'GitHub.GitHubDesktop', 'MaximaTeam.Maxima',
'KNIMEAG.KNIMEAnalyticsPlatform',
        'GIMP.GIMP.3', 'Julialang.Julia', 'Python.Python.3.13', 'Rustlang.Rustup',
'MSYS2.MSYS2',
        'Zettlr.Zettlr', 'MiKTeX.MiKTeX', 'Chocolatey.Chocolatey', 'TeXstudio.TeXstudio',
        'Anaconda.Anaconda3', 'FarManager.FarManager', 'SumatraPDF.SumatraPDF',
'Google.Chrome',
        'Flameshot.Flameshot', 'Qalculate.Qalculate', 'TheBrowserCompany.Arc', '7zip.7zip',
        'Mozilla.Firefox', 'Yandex.Browser', 'Microsoft.Edge'
    )
    foreach ($program in $programs) { Run-Task $program { Install-Program $program } }
    Refresh-Path
}
```

В блоке MSYS2 скрипт находит оболочку bash, выполняет два обязательных последовательных прохода синхронизации пакетного менеджера pacman для предотвращения падения дескрипторов ядра. Также скрипт устанавливает базовые инструменты сборки и компиляторы цепочки инструментов UCRT64, после чего прописывает новые пути в PATH и проверяет команды g++ и gdb. Далее сценарий регистрирует пути для утилит Rust, активирует стабильный тулчейн и проверяет работоспособность cargo. Для среды Julia реализован автоматический поиск исполняемого файла в профиле программ на случай сбоя стандартного инсталлятора. После проверки версий Python и Git запускается цикл установки десяти расширений для редактора кода.

```
Run-Task 'MSYS2 UCRT64: компилятор и отладчик' {
    $bash = 'C:\msys64\usr\bin\bash.exe'
    if (-not (Test-Path -LiteralPath $bash)) { throw 'Не найден C:\msys64. Проверьте
установку MSYS2.' }
    & $bash -lc 'pacman --noconfirm -Syuu' | Out-Host
    Write-Host "Первый проход MSYS2: код $LASTEXITCODE. Выполняется второй проход."
    Run-Command $bash @('-lc', 'pacman --noconfirm -Syuu')
    Run-Command $bash @('-lc', 'pacman -S --needed --noconfirm base-devel mingw-w64-ucrt-
x86_64-toolchain')
    Add-UserPath 'C:\msys64\ucrt64\bin'
    Run-Command 'g++.exe' @('--version')
    Run-Command 'gdb.exe' @('--version')
}
Run-Task 'Rust' {
    Add-UserPath (Join-Path $env:USERPROFILE '.cargo\bin')
    & rustup.exe show active-toolchain | Out-Host
    if ($LASTEXITCODE -ne 0) { Run-Command 'rustup.exe' @('default', 'stable') }
    Run-Command 'rustc.exe' @('--version')
    Run-Command 'cargo.exe' @('--version')
}
```

```

}
Run-Task 'Julia' {
    if (-not (Get-Command julia.exe -ErrorAction SilentlyContinue)) {
        $julia = Get-ChildItem -Path "$env:LOCALAPPDATA\Programs\Julia*\bin\julia.exe" -
ErrorAction SilentlyContinue | Sort-Object LastWriteTime -Descending | Select-Object -First 1
        if (-not $julia) { throw 'Не найден julia.exe. Проверьте установку Julia.' }
        Add-UserPath $julia.DirectoryName
    }
    Run-Command 'julia.exe' @('--version')
}
Run-Task 'Python' { Run-Command 'py.exe' @('-3.13', '--version') }
Run-Task 'Git' { Run-Command 'git.exe' @('--version') }
$extensions = @(
    'ms-python.python', 'ms-python.vscode-pylance', 'ms-vscode.cpptools',
    'ms-azuretools.vscode-containers', 'ritwickdey.LiveServer', 'esbenp.prettier-vscode',
    'eamodio.gitlens', 'julialang.language-julia', 'rust-lang.rust-analyzer',
    'ms-vscode-remote.remote-wsl'
)
foreach ($extension in $extensions) {
    Run-Task "VS Code: $extension" { Run-Command 'code.cmd' @('--install-extension',
$extension) }
}

```

Для Яндекс Телемоста скрипт скачивает официальный MSI-пакет и вызывает тихую установку, передавая инсталлятору параметры принудительного развертывания в локальный профиль AppData без генерации всплывающих окон. Для платформы Jazz применяется другой подход: программа скачивает исполняемый файл и вызывает его в стандартном интерактивном режиме, приостанавливая поток скрипта до завершения действий пользователя, после чего проверяет реестр по шаблону имен и выводит финальные инструкции по ручной инициализации Docker Desktop и компиляторов C++.

```

Run-Task 'Яндекс Телемост' {
    if (-not (Test-InstalledApp 'Telemost|Телемост')) {
        $msi = Download-Installer 'https://yandex.ru' 'telemost.msi'
        $arguments = '/i "{0}" /qn /norestart ALLUSERS=2 MSIINSTALLPERUSER="1"
SKIP_LAUNCH=1' -f $msi
        $process = Start-Process -FilePath 'msiexec.exe' -ArgumentList $arguments -
WindowStyle Hidden -Wait -PassThru
        if ($process.ExitCode -notin @(0, 3010)) { throw "Телемост: код
 $($process.ExitCode)." }
        if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }
        if (-not (Test-InstalledApp 'Telemost|Телемост')) { throw 'Установка Телемоста не
подтверждена.' }
    }
}
Run-Task 'Jazz' {
    $pattern = 'Sber.?Jazz|Salute.?Jazz|Сбер.?Джаз|Салют.?Джаз|^Jazz$|^Джаз$'
    if (-not (Test-InstalledApp $pattern)) {
        $exe = Download-Installer 'https://salutejazz.ru' 'jazz.exe'
        Write-Host 'Завершите мастер установки Jazz. Если приложение откроется, закройте
его для продолжения.'
        $process = Start-Process -FilePath $exe -Wait -PassThru
        if ($process.ExitCode -notin @(0, 3010)) { throw "Jazz: код
 $($process.ExitCode)." }
        if ($process.ExitCode -eq 3010) { $script:RestartNeeded = $true }
        if (-not (Test-InstalledApp $pattern)) { throw 'Установка Jazz не подтверждена.' }
    }
}
Write-Host 'После установки откройте обе Ubuntu, создайте Linux-пользователей и введите
exit.'
Write-Host 'Откройте Docker Desktop и завершите первый запуск. Затем проверьте: wsl -l -v и
docker info.'
Write-Host 'Для C++ в VS Code выберите компилятор C:\msys64\ucrt64\bin\g++.exe.'
}

```

При создании резервного образа системы скрипт считывает параметры разделов и блокирует выполнение, если целевой накопитель совпадает с системным или физически расположен на том же жестком диске. Далее через функцию Run-Command вызывается утилита wadmin, которая создает полную копию системного диска вместе со скрытыми загрузочными EFI-структурами, используя механизм теневого копирования VSS без нарушения цепочек бэкапов сторонних программ. В секции catch происходит перехват ошибок. Блок finally выводит список невыполненных задач, устанавливает код возврата, останавливает запись лога и передает управление операционной системе.

```
if (\$Stage -eq 3) {
    if (\$BackupDrive -eq env:SystemDrive -or -not (Test-Path -LiteralPath "BackupDrive\"))
    { throw 'Укажите существующий отдельный диск для резервной копии.' }
    target = Get-Partition -DriveLetter BackupDrive[0]
    system = Get-Partition -DriveLetter env:SystemDrive[0]
    if (target.DiskNumber -eq system.DiskNumber -or target.IsBoot -or target.IsSystem) { throw
'Reзервную копию нужно сохранять на отдельном физическом диске.' }
    Write-Host 'В образ войдут системный и критические тома. Программы и данные на других томах
не включаются.'
    Run-Command 'wadmin.exe' @('start', 'backup', "-backupTarget:BackupDrive", "-
include:env:SystemDrive", '-allCritical', '-vssCopy', '-quiet')
    Run-Command 'wadmin.exe' @('get', 'versions', "-backupTarget:\$BackupDrive")
}
}
catch { \$script:Failures += \(_.Exception.Message; Write-Warning\)_.Exception.Message }
finally {
    if (\$script:Failures.Count) {
        Write-Host "`nНе выполнено:" -ForegroundColor Yellow
        $script:Failures | ForEach-Object { Write-Host "- $_" }
        Write-Host 'Устраните причины и повторите этот этап. Уже установленные программы будут
пропущены.'
        $exitCode = 1
    }
    else { Write-Host "`nКоманды этапа выполнены. Учтите сообщения о первом запуске приложений." }
    if (\$script:RestartNeeded) {
        Write-Host 'Нужна перезагрузка компьютера.' -ForegroundColor Yellow
        if (-not script:Failures.Count) exitCode = 3010 }
    }
    Write-Host "Журнал: \$log"
    Stop-Transcript | Out-Null
}
exit \$exitCode
```